Plant Waterer: Design Spec

Date: 2026-10-16
Status: Approved design, ready for implementation planning
Project folder: `greenh`

---

1. Goal and scope

A hobby prototype of an automatic plant waterer. The software should be built well enough that other people could use it later, and it should stay easy to maintain.

Must-have: for each pot, tell the user whether the plant needs water and how much.

For now:

- Software only. Hardware signals are simulated.
- The UI is a web dashboard on the PC.

Later (out of scope for this spec):

- Real hardware: an Arduino Uno over USB serial, then ESP32 Wi-Fi nodes.
- A Raspberry Pi running the same backend (for learning).
- Actual watering hardware (pump or valve). The design leaves room for it but doesn't depend on it.
- Notifications.
- A native Swift iPhone app holding the data on the phone. This is a planned rewrite; the HTTP API (`openapi.yaml`) is the contract carried over.
- User accounts and login.

Principles

- **User data stays with the user.** All storage is local (SQLite). Nothing is hosted by the developer.
- **Only what's needed leaves the machine.** The LLM receives the pot profile and a reading summary, with no names, locations, or identifiers.
- **The LLM is replaceable.** DeepSeek is the first configured provider, not a dependency.
- **The simulator behaves exactly like real hardware,** so swapping it out doesn't change the backend.

Definition of done for the prototype

1. The simulator runs 2–3 pots on one node, and readings flow into the backend.
2. The dashboard shows each pot's current values, history charts, and the latest advice.
3. Both "Check now" and scheduled checks work.
4. Logging a watering visibly raises moisture and changes the next recommendation.
5. Everything starts with one `docker compose up`.

---

2. Hardware (future reference; drives the simulation)

Prices are rough AliExpress-to-Amazon ranges in USD and were not checked against current listings.

Core sensor set (v1)

Sensor	Measures	Interface	Cost	Difficulty	Notes
Capacitive soil moisture sensor (v1.2/v2.0)	Soil water content	Analog, 3 wires	$1–3	Very easy	One per pot. Avoid resistive probes (they corrode). Needs dry/wet calibration. Buy 3–5; some boards are faulty.
BME280 or AHT20	Air temperature and humidity	I²C, 4 wires	$2–6	Easy	Better than DHT11. Shared per node.
BH1750	Light (lux)	I²C, 4 wires	$1–3	Easy	Shares the I²C bus. Shared per node.

About $5–12 per spot plus the board. Breadboard and jumper wires; some modules need header pins soldered (about 10 minutes).

Optional extras (later)

- Soil temperature: DS18B20 waterproof probe ($2–4, easy).
- Pot weight: load cell + HX711 ($5–10, medium). Most accurate water measure, fiddly to calibrate.
- Water tank level: float switch ($1–3). Only with a pump.
- Watering: 5 V mini pump + MOSFET or relay + tubing ($5–10).

Boards

- **Arduino Uno** (owned): prototyping over USB serial. 6 analog inputs, so one board covers a windowsill.
- **ESP32 dev board** ($5–8): Wi-Fi node that posts readings over HTTP. Programmed with the same Arduino tooling.
- **Raspberry Pi 4/5:** runs the backend later.

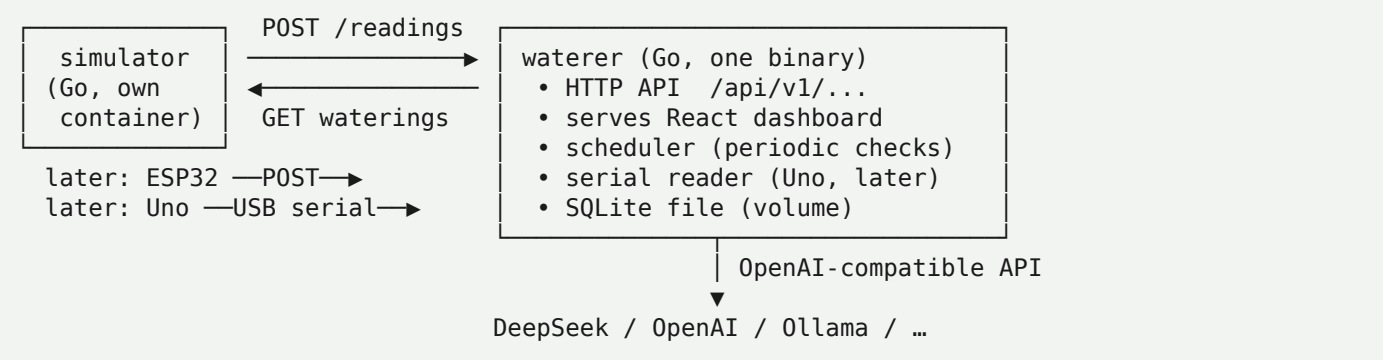
Firmware on the Uno/ESP32 is C++ (Arduino sketch).

3. Stack

Part	Choice
Backend	Go, single static binary (`CGO_ENABLED=0`)
Storage	SQLite via `modernc.org/sqlite` (pure Go), WAL mode, migrations with `goose`

Part	Choice
Dashboard	React + TypeScript, Vite, TanStack Query, React Router, Recharts, Tailwind
API contract	OpenAPI (`api/openapi.yaml`); Go types via `oapi-codegen`, TS types via `openapi-typescript`
LLM	Any OpenAI-compatible API; DeepSeek first
Logging	`log/slog`, structured JSON
Packaging	Docker, multi-stage, distroless base, multi-arch (`amd64`, `arm64`)
Serial (later)	`go.bug.st/serial`

4. Architecture



The simulator plays the plants and nothing more. It learns about waterings by polling the backend's watering log. The backend contains no simulation code.

Hardware connection

There is one ingestion path: `POST /api/v1/readings`.

- The simulator and ESP32 nodes post to it directly.
- The Uno (later) prints JSON lines over USB serial. A serial reader inside the backend turns each line into the same internal `Ingest` call.

Backend layout (one Go module)

Package	Responsibility
`core/`	Domain types (Node, Pot, Reading, Watering, Advice), summarizer, `Advisor` interface, rule-based advisor, calibration, offline detection. Pure logic, no I/O.
`advisor/openaicompat/`	Prompt building, HTTP client, response parsing and validation.
`store/sqlite/`	Storage implementing interfaces defined in `core`.
`ingest/`	`Ingest(reading)` service. HTTP handler and serial reader are thin callers.

Package	Responsibility
`scheduler/`	Per-pot periodic advice checks.
`api/`	HTTP handlers and JSON types generated from `openapi.yaml`.
`web/`	React app (Vite), built and embedded with `go:embed`.
`cmd/waterer`, `cmd/simulator`	Entry points.

5. Data model and data flow

Nodes, channels, and pots

- A **node** is one board. It reports shared ambient readings (temperature, humidity, light) and one or more **channels**.
- A **channel** is one soil moisture input on that board.
- A **pot** is linked to exactly one `(node_id, channel)`. Moving a sensor to another plant means changing the link in the dashboard, not reflashing.

Reading payload

```
POST /api/v1/readings
{
  "node_id": "windowsill-1",
  "ts": "2026-10-16T14:05:00Z",
  "ambient": { "temp_c": 22.4, "humidity_pct": 48, "light_lux": 3200 },
  "channels": [ { "ch": 0, "moisture_raw": 512 }, { "ch": 1, "moisture_raw": 389 } ]
}
```

- `ts` is optional; the backend uses receive time if missing.
- `ts` may be up to 7 days in the past (for backfill and buffered resends) and at most 5 minutes in the future.

Flow

1. **Readings arrive** every interval (5 minutes by default; faster in the simulator).
2. **Ingest validates and stores them raw.** An unknown `node_id` is auto-registered as a new node with unlinked channels.
3. **Raw moisture becomes a percentage on read,** using the pot's calibration (dry and wet raw values). Re-calibrating fixes past history too.
4. **The scheduler** wakes every minute, finds pots due for a check, builds a summary, calls the advisor, and stores an `Advice` record with provider, model, prompt version, fallback flag, and the exact input summary.
5. **"Check now"** runs the same path synchronously and returns the new advice.
6. **No data cleanup in v1.** Volume is trivial for SQLite; a rollup job can be added later.

Pot record

- Profile: name, species, category (succulent / foliage / flowering / herb), pot volume (ml), soil type, placement.

- Link: `node_id`, `channel`.
- Calibration: `dry_raw`, `wet_raw` (empty until calibrated).
- Rules: moisture threshold %, amount as fraction of pot volume (prefilled from category, editable).
- Check interval (default 6 hours).

Watering record

- Pot, timestamp, amount (ml), optional source (manual now; pump later).

---

6. API (`/api/v1`, REST/JSON)

Defined in `api/openapi.yaml`, which is also the contract for the future Swift app.

Area	Endpoints
Ingestion	`POST /readings`
Nodes	`GET /nodes`, `PATCH /nodes/{id}` (rename)
Pots	`GET /pots`, `POST /pots`, `GET /pots/{id}`, `PATCH /pots/{id}`, `DELETE /pots/{id}`
History	`GET /pots/{id}/readings?from=&to=&step=` (optional downsampling for charts)
Waterings	`GET /pots/{id}/waterings`, `POST /pots/{id}/waterings`, `GET /waterings?since=` (includes `node_id` and `channel`; used by the simulator)
Advice	`GET /pots/{id}/advice` (latest + history), `POST /pots/{id}/advice` (“Check now”)
System	`GET /health`

All errors use one shape: `{ "code": "...", "message": "..." }`.

Advisor settings (provider, base URL, model, API key) come from environment variables or a config file, never from the API, so the key never reaches the browser.

---

7. Advisor

Pre-checks (in `core`, no LLM)

Condition	Result `status`
No reading for the pot’s channel within `OFFLINE_AFTER` (default 30 min, configurable)	`sensor_offline`
Pot not calibrated	`needs_calibration`
Otherwise	`ok` (carries a recommendation)

Input summary (the only thing sent to the LLM)

- **Pot:** species, category, pot volume (ml), soil type, placement.
- **Now:** moisture %, air temperature, humidity, light.
- **Trend:** hourly moisture over the last 48 h, drying rate (% per day), light total (lux-hours) for the last 24 h.
- **Last watering:** time, amount, moisture before and after.
- **Data quality notes:** for example “gap of 3 h in readings”.

Output (same shape for every provider and the rules)

```
        { "needs_water": true, "amount_ml": 250, "confidence": "medium", "reason": "Moisture
18%, dropping ~9%/day; Monstera prefers 30-50%." }
```

Prompt and validation

- Fixed system prompt (role, rules, JSON schema) plus the summary as the user message. The prompt lives in one versioned file; each advice record stores the prompt version.
- Our code validates every answer:
  - valid JSON
  - `confidence` ∈ {low, medium, high}
  - `needs_water: false` ⇒ `amount_ml` = 0
  - 0 ≤ `amount_ml` ≤ half the pot volume
  - `reason` is short
- On timeout (30 s), HTTP error, or invalid answer: one retry, then the rule-based advisor answers and the record is flagged `fallback` with the reason.

Rule-based advisor

- Uses the pot’s moisture threshold and amount fraction (prefilled from category).
- Runs on **every** check, even when the LLM answers. Its result is stored next to the LLM’s. If they strongly disagree, the dashboard shows an “LLM and rules disagree” marker.
- Fully offline, always available.

Providers

- `Advisor` interface in `core`; the rest of the system never knows which model answered.
- **`openaicompat` provider:** configured with `base_url`, `model`, `api_key`. Covers DeepSeek, OpenAI, OpenRouter, and local servers (Ollama, LM Studio, llama.cpp). Switching is a config change.
- **`rules` provider:** offline mode, no LLM.
- Other APIs (native Anthropic, Gemini) can be added as small adapters later.
- No provider-specific tricks: plain JSON prompt, validated by our code.

Config: `ADVISOR_PROVIDER` (`openaicompat` | `rules`), `ADVISOR_BASE_URL`, `ADVISOR_MODEL`, `ADVISOR_API_KEY`.

---

8. Simulator

A separate Go program (`cmd/simulator`) in its own container. It behaves like an ESP32 node with real soil attached.

Config (YAML)

```yaml
        backend: http://waterer:8080
        interval: 30s
        speed: 60            # drying and day cycle 60× faster; timestamps stay real
        backfill: 48h        # at startup, post 48h of past readings so trends exist
immediately
        nodes:
          - id: windowsill-1
            channels:
              - { ch: 0, category: foliage,   pot_ml: 2000, start_moisture: 45 }
              - { ch: 1, category: succulent, pot_ml: 500,  start_moisture: 20 }
              - { ch: 2, category: herb,      pot_ml: 800,  start_moisture: 60 }
```

Environment model (per node)

- **Light:** day/night curve, zero at night, a few thousand lux at midday, random cloud dips.
- **Temperature:** follows the day curve, about 19–25 °C.
- **Humidity:** moves opposite to temperature.

Soil model (per channel)

- Hidden “true” moisture %.
- **Drying:** each tick drops by a base rate (from category and pot size), scaled up by heat, light, and dry air, scaled down at night.
- **Watering:** polls `GET /waterings?since=`. A watering raises moisture in proportion to amount ÷ pot volume, overshoots, then settles over about an hour as excess drains.
- **Sensor output:** true % converted to a raw value like a capacitive sensor: inverted (drier = higher raw), per-channel fake dry/wet range, ±1–2% noise, integers. This exercises backend calibration for real.

Time

- `speed` compresses the physics only; timestamps stay real, so the scheduler, offline detection, and watering log need no fake clocks.
- `backfill` posts historical readings once at startup.

Failure modes (switchable in config)

- `dropout`: node goes silent → should show “sensor offline”.
- `stuck`: one channel repeats the same value.
- `out_of_range`: garbage raw values → ingest rejects.
- Backend unreachable: simulator buffers and resends later (same behavior wanted in ESP32 firmware).

Not in v1: serial output mode (JSON lines on a virtual serial port). Added together with the backend serial reader.

---

9. Dashboard

React + TypeScript + Vite. TanStack Query polls every 30 s (no WebSockets in v1). Vite dev server proxies `/api` to the backend; release builds are embedded in the Go binary. No login, metric units.

Screens

1. **Overview:** one card per pot with name and species, moisture gauge, status badge (*OK*, *Water ~250 ml*, *Sensor offline*, *Needs calibration*), time of last check, and the disagreement marker when relevant.
2. **Pot detail:**
   - Charts over 24 h / 7 d / 30 d: moisture (with watering and advice markers), temperature and humidity, light.
   - Latest advice: reason, provider and model, fallback flag.
   - **Check now** button.
   - **Log watering** button, amount prefilled from the latest advice.
   - Advice history; each entry expands to show the exact summary sent to the advisor.
3. **Pot settings:** profile, rule thresholds and check interval, node/channel link, and a **calibration wizard** (“sensor in air → Capture dry”, “sensor in water → Capture wet”, using the channel’s latest raw reading).
4. **Nodes:** each node with last-seen time. Unlinked channels get a **“Create pot from this channel”** button.

---

10. Error handling

Area	Situation	Behavior
Ingest	Invalid payload	`400` with field-level details
Ingest	Channel not linked to a pot	Store anyway; waits for linking
Ingest	Duplicate `(node_id, ts)`	Ignore silently (safe resends)
Advisor	Timeout, HTTP error, invalid answer	One retry, then rules fallback; flagged with reason
Scheduler	One pot’s check fails	Pots independent; others not blocked
Scheduler	Check already running for a pot	Never two concurrent checks per pot
Scheduler	Overdue after downtime	One check per overdue pot, no burst
Startup	Invalid config	Stop with a clear message
Startup	No LLM API key	Start in `rules` mode, log a warning
Storage	Schema changes	SQLite WAL; `goose` migrations embedded, run on startup
API	Any error	`{ "code", "message" }`
Logging	Visibility	`slog` JSON: advisor latency, fallback reasons, ingest rejections; API keys never logged
Dashboard	Stale data	“Last reading 2 h ago” hint per pot
Dashboard	Failed request	Non-blocking error toast

---

11. Testing

No real LLM calls in automated tests.

Layer	What's tested	How	In `make test`
`core`	Summarizer, rules, answer validation, calibration, offline detection	Table-driven unit tests (most coverage)	Yes
`advisor/openaicompat`	Good answers, bad JSON, out-of-range amounts, timeouts, HTTP 500	`httptest` fake server with canned responses; optional real call with `-tags live`	Yes (live: no)
`store/sqlite`	Queries, migrations, duplicates	Real SQLite in a temp file	Yes
`api`	Every endpoint end to end	`httptest`, temp DB, fake advisor, checked against `openapi.yaml`	Yes
simulator	Soil model dries, watering raises moisture, stays in 0–100%	Unit tests	Yes
whole system	`compose up` → backfill → advice exists → watering raises moisture	Smoke test, `-tags e2e`	No
dashboard	Type checking; pot card and calibration wizard	`tsc`, Vitest, Testing Library	Yes

A GitHub Actions workflow running `make test` can be added once the repo is on GitHub.

12. Deployment

- **Dockerfile (multi-stage):** build React with Node → build Go with `CGO_ENABLED=0` → copy onto a distroless base. Image roughly 20–30 MB.
- **Multi-arch:** `docker buildx` produces `amd64` (laptop) and `arm64` (Pi) images.
- **`docker-compose.yml`:** services `waterer` and `simulator`; a volume for the SQLite file; `.env` for advisor settings.
- **Raspberry Pi (later):** same compose file without the simulator, plus `/dev/ttyACM0` device mapping once the Uno is connected.

13. Future work (not in this spec)

- Serial reader + simulator serial mode; Uno firmware.
- ESP32 firmware posting to `/readings`, with buffering.
- Pump control and automatic watering.
- Trigger-based checks (moisture threshold crossings).
- Notifications.
- Soil temperature and pot weight sensors.
- Data rollup/cleanup.
- Native Swift iPhone app (client generated from `openapi.yaml` with `swift-openapi-generator`).